

Book Recommender

Manuale Tecnico

Gianmarco Maffioli, Francesca Rolla, Gabriele Fabbian

1. Introduzione	2
2. Premessa	2
3. Software Design	2
3.1 Struttura dell'applicazione.....	3
3.2 Connessione al database	5
3.3 Traduzione entità relazionali in entità Java	7
3.4 Architettura	8
3.4.1 Persistence Layer.....	10
3.4.2 Data Access Layer.....	11
3.4.3. Service Layer.....	12
3.4.4 Presentation Layer	13
3.5 Strutture dati	14
3.6 Design Pattern.....	15
4. Algoritmi	17
4.1 Ricerca per titolo o autore	17
4.2 Ricerca per autore e anno.....	18
4.3 Ricerche con token utente	18
4.4 Recupero dettagli libro	19
5. Gestione della concorrenza	19
6. Logging.....	20
6.1 Client	20
6.2 Server.....	20
7. Strumenti, librerie e linguaggi utilizzati	20
8. Limiti dell'applicazione e conclusioni.....	21
9. Bibliografia.....	22

1. Introduzione

“*Book Recommender*” è un sistema per la valutazione e raccomandazione di libri, in grado di permettere agli utenti registrati di inserire recensioni e a tutti gli utenti di consultare le valutazioni e ricevere consigli di lettura. Per ulteriori informazioni sul funzionamento del programma, incluse le istruzioni per l’avvio, consultare il *Manuale Utente*.

2. Premessa

Sebbene l’applicativo sia stato prodotto nell’ambito del progetto del *Laboratorio B* per il corso di laurea in informatica dell’*Università degli Studi dell’Insubria*, si è cercato di impiegare l’organizzazione strutturale tipica delle applicazioni di livello *entreprise*, adottandone il più possibile.

La documentazione non descrive ogni singola classe, ma i componenti fondamentali per comprendere funzionamento e manutenzione, inoltre questo manuale comprende una parte limitata dei diagrammi UML, fare riferimento alla cartella dedicata per le versioni complete.

Per i dettagli completi sulle classi si rimanda alla *Javadoc* allegata.

3. Software Design

Lo sviluppo dell’applicazione è stato condotto utilizzando Java 17, una versione stabile e ampiamente utilizzata del linguaggio che integra numerose innovazioni orientate a ridurre il codice ridondante e a migliorare la leggibilità e la manutenibilità del software. L’intero sistema è stato progettato e implementato in Java 17 ed è stato sottoposto a test su piattaforme Windows (10 e 11), Linux (Ubuntu 24.04.1 LTS), MacOS-aarch64 (Sonoma 14.6.1) e MacOS-Intel (Sonoma 14.3.1).

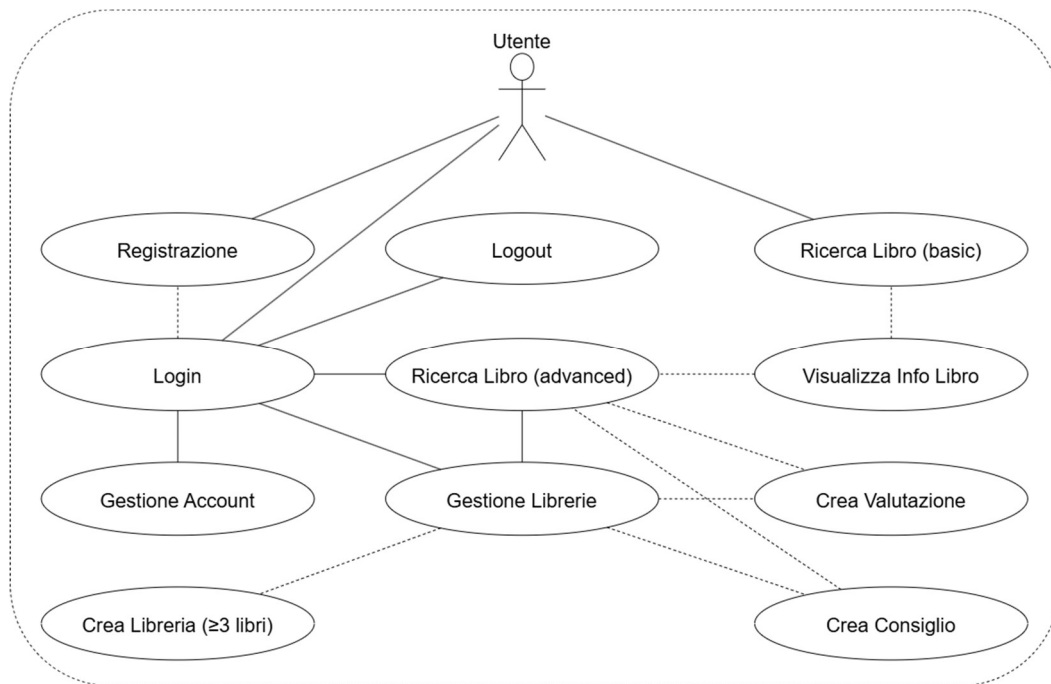
Per quanto concerne la gestione dei dati, è stato adottato il DBMS *PostgreSQL*, in linea con le specifiche fornite. La scelta di questo sistema ha consentito di garantire robustezza, affidabilità e conformità agli standard SQL. La comunicazione tra client e server è stata realizzata mediante l’impiego della tecnologia RMI (Remote Method Invocation), la quale ha permesso di semplificare la logica di connessione e di rendere trasparente la gestione del multi-threading. Tale approccio si è rivelato efficace per la gestione simultanea di più client, permettendo al contempo di concentrare l’attenzione sulla logica applicativa piuttosto che sugli aspetti infrastrutturali. L’interfaccia grafica

del client è stata sviluppata con JavaFX, strumento che garantisce una netta separazione tra la logica applicativa e la componente di presentazione. Questa scelta ha contribuito a incrementare la modularità del sistema e a facilitare sia la manutenzione che l'estensione futura dell'applicazione, oltre a migliorare la leggibilità del codice.

3.1 Struttura dell'applicazione

Il progetto è organizzato in tre moduli principali, rispettando la richiesta del committente di un'applicazione Client-Server:

- **client**: applicazione JavaFX con GUI, controller e viste FXML.
- **common**: classi condivise (modelli, interfacce RMI).
- **server**: logica lato server, connessione DB e servizi RMI, con una piccola integrazione grafica per facilitarne l'uso.



Use Case Diagram Semplificato



Struttura del progetto

La struttura attualmente adottata per il progetto segue il modello predefinito dei progetti Maven. L'uso di Maven consente di beneficiare di una configurazione standardizzata e ampiamente riconosciuta, che facilita sia la fase di sviluppo che quella di integrazione continua. Grazie alla presenza dei file pom.xml, le dipendenze vengono gestite automaticamente e il progetto si auto-configura, garantendo portabilità e riproducibilità dell'ambiente di build senza interventi manuali complessi.

Oltre alle cartelle dedicate ai file sorgenti Java (*src/main/java*), nei moduli client e server è presente anche la directory resources, che contiene tutti gli asset non strettamente legati alla logica applicativa ma fondamentali per l'interfaccia e l'esperienza utente. In particolare:

- **fxml**: i file che descrivono la struttura e la composizione delle viste dell'interfaccia grafica.
- **icons**: immagini e icone utilizzate nelle schermate.

- **stylesheets (CSS):** fogli di stile per la personalizzazione grafica delle interfacce JavaFX.

Questa suddivisione consente di mantenere una chiara separazione tra logica applicativa e componenti di presentazione, rispettando le buone pratiche di modularità e manutenibilità del software.

3.2 Connessione al database

La connessione a PostgreSQL è gestita tramite HikariCP (connection pooling) e incapsulata nella classe DBManager, che inizializza e mantiene il DataSource del pool ed espone connessioni sicure ai DAO. La configurazione (porta, user, password, nome DB) viene inserita dall'utente all'avvio del server.

```
public boolean isTcpPortAvailable(int portNumber) {
    try (ServerSocket ss = new ServerSocket(portNumber)) {
        ss.setReuseAddress(true);
        logger.info("Test sulla porta TCP " + portNumber + " riuscito.");
        return true;
    } catch (IOException e) {
        logger.log(Level.WARNING, "Test sulla porta TCP " + portNumber + "
fallito: " + e.getMessage());
        return false;
    }
}
```

Il metodo *isTcpPortAvailable* (int portNumber) ha la funzione di verificare se una determinata porta TCP è libera e quindi utilizzabile dal server prima di avviarne i servizi. Viene istanziato un oggetto *ServerSocket* sul numero di porta specificato (portNumber).

Se la porta è disponibile, il costruttore riesce a creare correttamente il socket.

L'uso del *try-with-resources* garantisce che il socket venga chiuso automaticamente al termine del blocco, indipendentemente dall'esito della prova.

Questa chiamata abilita il riuso dell'indirizzo locale dopo la chiusura del socket, evitando che il test lasci la porta "bloccata" nello stato TIME_WAIT. In altre parole, assicura che la verifica non impedisca un successivo avvio effettivo del server sulla stessa porta.

Se l'operazione ha successo, viene registrato un messaggio di log a livello INFO e il metodo restituisce true, indicando che la porta è disponibile.

Se si verifica un'eccezione (IOException), ad esempio perché la porta è già in uso da un altro processo o non è permesso aprirla, viene registrato un messaggio di log a livello WARNING e il metodo restituisce false.

```

public boolean connect(String url, String user, String password) {
    if (dataSource != null && !dataSource.isClosed()) {
        logger.log(Level.WARNING, "Pool già inizializzato.");
        return true;
    }
    try {
        HikariConfig config = getHikariConfig(url, user, password);
        dataSource = new HikariDataSource(config);
        logger.log(Level.INFO, "HikariCP pool creato con successo.");
        return true;
    } catch (Exception e) {
        logger.log(Level.SEVERE, "Errore inizializzazione HikariCP", e);
        return false;
    }
}

private static HikariConfig getHikariConfig(String url, String user, String
password) {
    HikariConfig config = new HikariConfig();
    config.setJdbcUrl(url);
    config.setUsername(user);
    config.setPassword(password);
    config.setMaximumPoolSize(20);
    config.setMinimumIdle(5);
    config.setIdleTimeout(60_000);
    config.setConnectionTimeout(30_000);
    config.setPoolName("BookRecommenderPool");
    config.addDataSourceProperty("cachePrepStmts", "true");
    config.addDataSourceProperty("prepStmtCacheSize", "100");
    config.addDataSourceProperty("prepStmtCacheSqlLimit", "2048");
    return config;
}

```

Il metodo *connect* (String url, String user, String password) ha la responsabilità di inizializzare in modo sicuro il pool di connessioni al DB tramite HikariCP.

Se *dataSource!=null* e *!dataSource.isClosed()*, il pool è già attivo e si evita una seconda inizializzazione (idempotenza), quindi ritorna *true*.

Vantaggio: nessun overhead di riconfigurazione, si prevengono leak e race condition di doppio avvio.

Si invoca *getHikariConfig*(url, user, password) per ottenere una HikariConfig completa e coerente con il profilo prestazionale desiderato.

dataSource = new HikariDataSource(config) crea il pool, inizializzando gli interni di *Hikari*.

In caso di successo si logga a INFO, in caso di errore si cattura l'eccezione, si logga a SEVERE e si ritorna *false*.

- **jdbcUrl, username, password**: credenziali di connessione. Se invalidi avviene il fallimento immediato in costruzione di HikariDataSource.
- **maximumPoolSize = 20**: limite superiore di connessioni contemporanee (hard cap). Indica il throughput massimo delle query concorrenti, un valore troppo basso genera contention (thread in attesa), troppo alto aumenta pressione sul DB e memory footprint.
- **minimumIdle = 5**: numero minimo di connessioni inattive mantenute “calde” dal pool. Riduce quindi la latenza quando più utenti sono connessi contemporaneamente.
- **idleTimeout = 60s**: connessioni oltre minimumIdle inattive per più di 60 s vengono chiuse. Si utilizza quindi un minor numero di risorse ma si aumenta la latenza quando aumentano le richieste.
- **connectionTimeout = 30s**: tempo massimo di attesa per ottenere una connessione. Se scade il chiamante riceve un’eccezione non bloccante, esso rimane in attesa in maniera indefinita fino a che il server non risponde.
- **poolName**: nome del pool per il logging/monitoring.
- **cachePrepStmts, prepStmtCacheSize, prepStmtCacheSqlLimit**: proprietà pass-through al driver JDBC per ottimizzare la cache dei prepared statement.

3.3 Traduzione entità relazionali in entità Java

Le principali entità del database (‘libri’, ‘librerie’, ‘valutazioni’, ‘consigli’) sono mappate in POJO serializzabili (‘Libro’, ‘Libro_Details’, ‘Valutazione’).

Le relazioni N:M (es. libri in librerie, consigli per utenti) vengono gestite da DAO con query SQL dedicate, piuttosto che da oggetti mappati direttamente.

© Valutazione
□ valutazioni: List<Float> □ serialVersionUID: long □ libro: Libro □ username: String □ commenti: List<String>
○ Valutazione(String, List<Float>, List<String>, Libro): ○ getIdLibro(): int ○ getUsername(): String ○ getValutazioni(): List<Float> ○ getCommenti(): List<String> ○ getLibro(): Libro ○ toString(): String

© Libro_Details
□ mGradevolezza: float □ mStile: float □ serialVersionUID: long □ mOriginalita: float □ valutazioni: List<Valutazione> □ mFinale: float □ mContenuto: float □ mEdizione: float □ consigli: Hashtable<String, List<Libro>>
○ Libro_Details(Hashtable<String, List<Libro>>, List<Valutazione>): ○ getmContenuto(): float ○ getmGradevolezza(): float ○ getmOriginalita(): float ○ getValutazioni(): List<Valutazione> ○ getmStile(): float ○ getConsigli(): Hashtable<String, List<Libro>> ○ getmEdizione(): float ○ getmFinale(): float

© Libro
□ serialVersionUID: long □ descrizione: String □ id: int □ editore: String □ titolo: String □ prezzo: float □ annoPubblicazione: short □ mesePubblicazione: short □ autore: String □ categoria: String
○ Libro(int, String, String, String, String, String, String, float, short, short): ○ equals(Object): boolean ○ getDescrizione(): String ○ hashCode(): int ○ getAutore(): String ○ getMesePubblicazione(): String ○ getCategory(): String ○ getId(): int ○ getTitolo(): String ○ getEditore(): String ○ getAnnoPubblicazione(): short ○ getPrezzo(): float ○ toString(): String

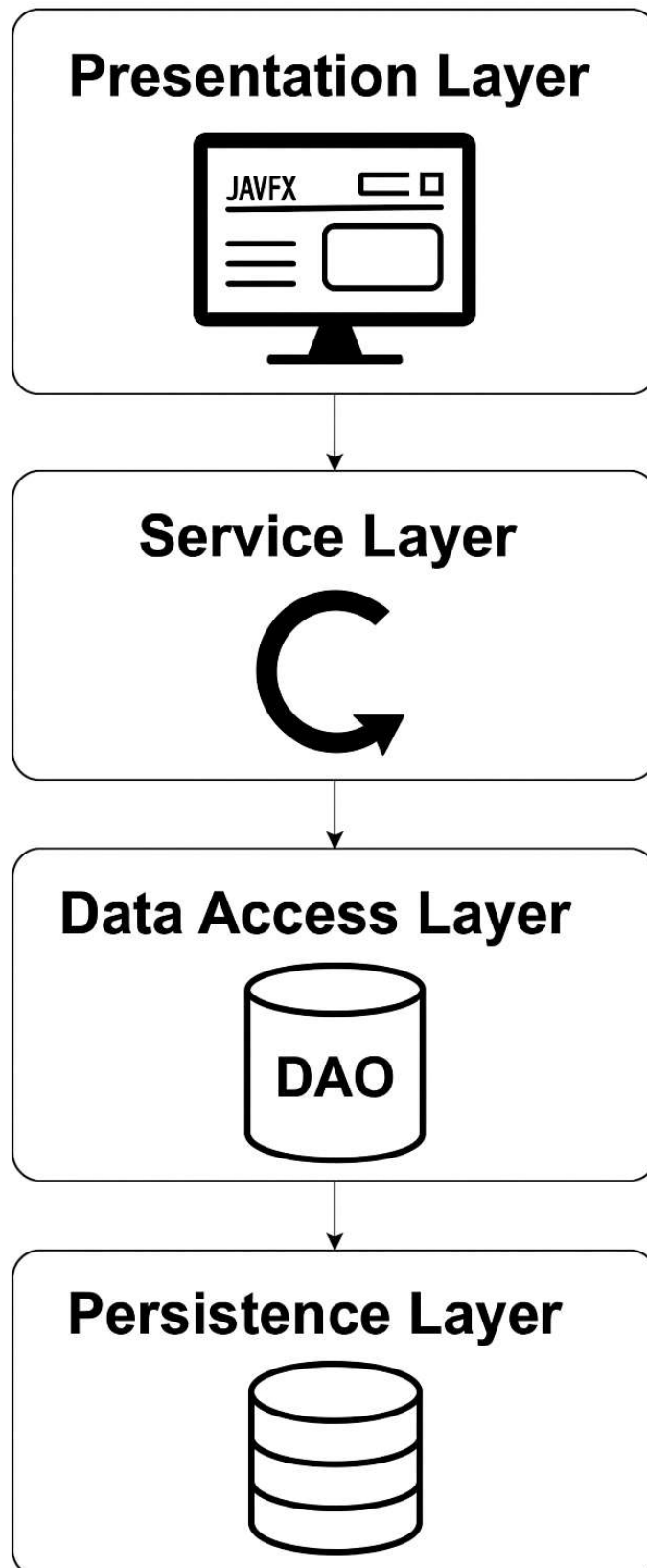
Queste classi presentano molte caratteristiche tipiche dei DTO (immutabilità logica e prevalenza di campi dati) e sarebbe stato possibile renderle dei *record*. Tuttavia, l'analisi del codice ha evidenziato alcune complicazioni: in particolare la necessità di mantenere metodi personalizzati (ad esempio *toString()* mirati per la UI, builder e logiche di confronto e calcolo come il voto medio in Valutazione) e la gestione di serializzazione RMI. Questi aspetti rendono meno pratico l'uso dei record, portando alla scelta consapevole di mantenere tali entità come classi ordinarie.

È stato invece deliberatamente scelto di non creare record per oggetti di tipo Libreria: in questo caso, la modellazione è stata risolta efficacemente tramite le strutture di collezione già presenti (*List<Libro>*), che permettono di rappresentare e gestire dinamicamente l'insieme dei libri di una libreria senza introdurre ulteriori entità dedicate. Questa scelta semplifica l'architettura e mantiene coerente l'uso delle collezioni nel presentation e nel service layer.

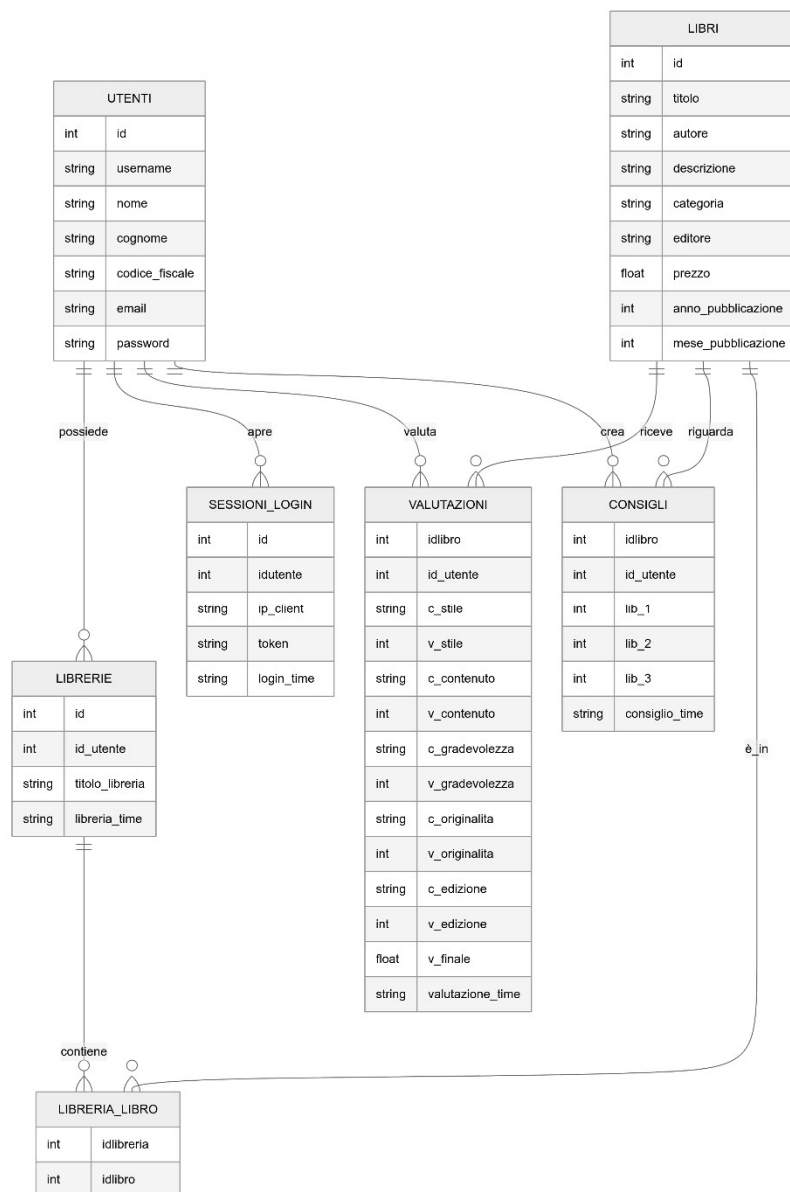
3.4 Architettura

Il sistema adotta un'architettura a 3 livelli + DB:

1. **Persistence Layer:** schema SQL definito in *`data/tablecreation.sql`*.
2. **Data Access Layer:** implementazioni per ogni entità.
3. **Service Layer:** validazione input e gestione eccezioni.
4. **Presentation Layer:** GUI JavaFX con FXML, controller e CSS.



3.4.1 Persistence Layer



Lo schema riflette bene i casi d'uso dell'app: le entità core (*libri*, *utenti*) sono distinte dalle azioni dell'utente (*valutazioni*, *consigli*, *librerie*) con un modello normalizzato e leggibile. L'integrità referenziale è chiara grazie alle FK esplicite e alla tabella di giunzione *libreria_libro* con PK composta e cancellazione a cascata, che riflette perfettamente la gestione delle librerie personali. La colonna generata *v_finale* in *valutazioni* incapsula l'algoritmo di calcolo direttamente nel database, garantendo coerenza tra persistenza e logica applicativa e semplificando le query. I vincoli di univocità su *username*, *email* e *codice_fiscale* rafforzano l'identità dell'utente, mentre i *timestamp* di default forniscono tracciabilità naturale per ordinamenti e audit (creazione librerie, consigli, sessioni). La tabella *sessioni_login* gestisce l'autenticazione

tramite token, facilitando sia l'integrazione lato service/RMI sia eventuali analisi operative.

3.4.2 Data Access Layer

Il DAL incapsula l'accesso a PostgreSQL, isolando il resto del server (servizi RMI) dai dettagli di persistenza.

Il pooling centralizzato tramite DBManager garantisce riuso efficiente delle connessioni. La configurazione (max pool size, timeouts, leak detection) bilancia throughput e latenza. L'inizializzazione unica evita over-provisioning e rende prevedibili i consumi.

Ogni servizio espone metodi focalizzati (es. fetch per chiavi, ricerche filtrate, inserimenti atomici), con parametri tipizzati e nessuna logica di presentazione. Questo riduce l'accoppiamento con la UI e facilita il riuso da parte di diversi servizi RMI.

```
public boolean isTokenNotValid(Token token) {
    String query = "SELECT 1 FROM SESSIONI_LOGIN WHERE TOKEN = ? AND IP_CLIENT = ?";
    try (Connection conn = getConnection();
        PreparedStatement stmt = conn.prepareStatement(query)) {
        stmt.setString(1, token.token());
        stmt.setString(2, token.ipClient());
        try (ResultSet rs = stmt.executeQuery()) {
            return !rs.next();
        }
    } catch (SQLException e) {
        logger.log(Level.SEVERE, "Errore nella validazione del token " +
            token.token() + " utente di id " + token.userId() + " IP:" + token.ipClient(),
            e);
        return true;
    }
}
```

Il caso in esempio è un metodo contenuto nella classe Singleton ServerUtil, ogni servizio può accedere a questo metodo per verificare se il token associato ad un determinato utente sia valido o meno.

L'uso sistematico di PreparedStatement garantisce sicurezza (niente SQL injection), caching dei piani lato server e prestazioni stabili.

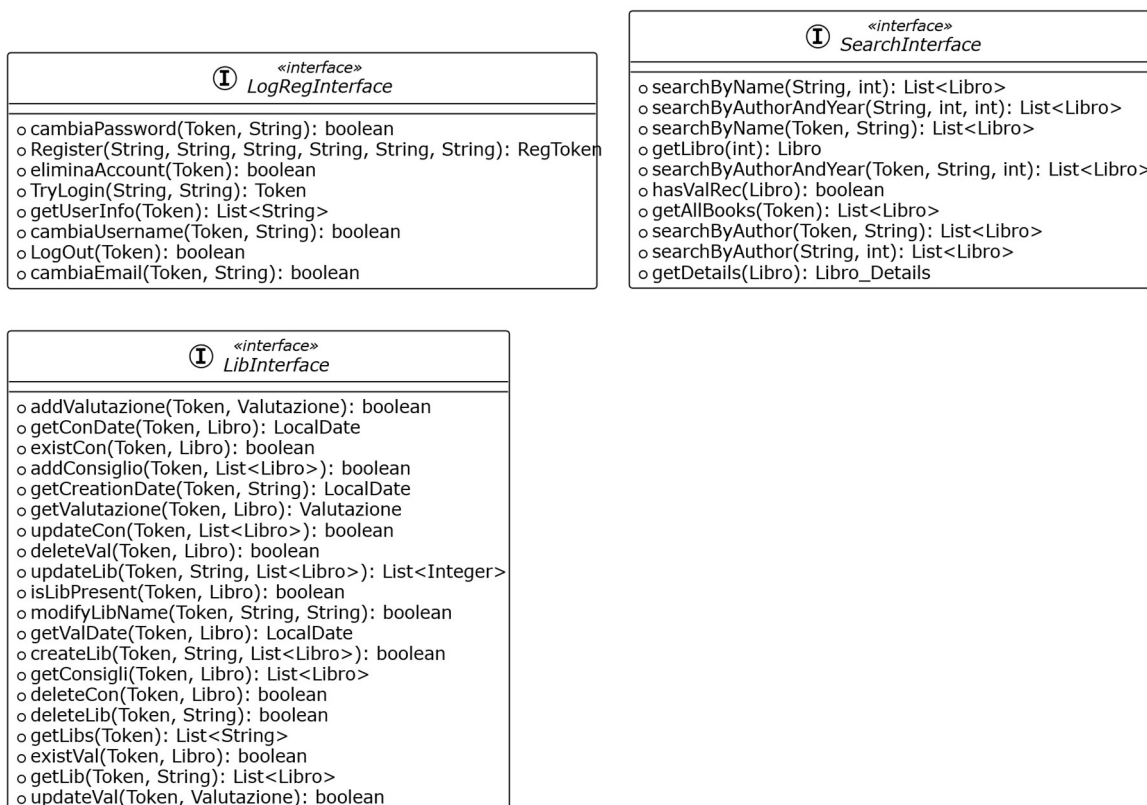
I record vengono mappati su oggetti del common (es. Libro, Libro_Details, Valutazione). I tipi numerici e le colonne generate (es. media finale in DB) riducono il rischio di discrepanze tra calcolo applicativo e dati persistiti.

Le operazioni che coinvolgono più tabelle (es. creare libreria + popolare associazioni) sono incapsulate in transazioni con commit/rollback esplicito. La gestione transazionale è ristretta al perimetro più piccolo possibile, per migliorare la concorrenza.

Le query di ricerca espongono parametri per LIMIT/OFFSET (o FETCH FIRST ... ROWS) e ordinamenti espliciti. Questo evita carichi eccessivi in rete e UI, e rende deterministico il comportamento dei risultati.

3.4.3. Service Layer

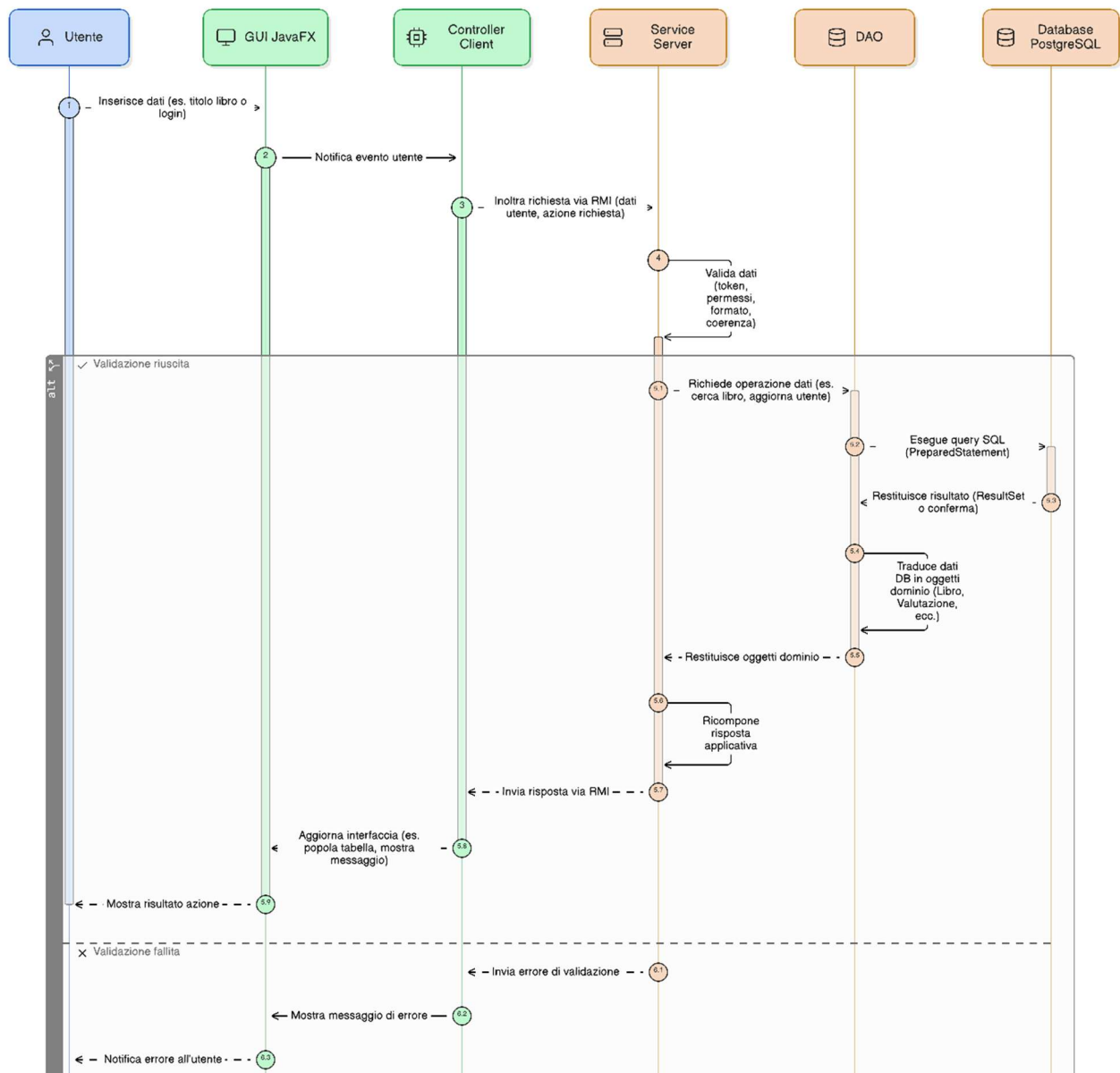
Il Service Layer espone i contratti RMI definiti in *common.interfaces* e li realizza tramite le classi (*InterfaceImpl*, *LogRegInterfaceImpl*, *SearchInterfaceImpl*, *LibInterfaceImpl*, *MonitorInterfaceImpl*). Ogni service è stateless e funge da orchestratore, riceve le richieste dal client, valida il *Token* e i parametri, applica le regole di dominio (es. flussi di login/registrazione, gestione librerie e valutazioni, consigli, vincoli funzionali), quindi delega le operazioni di persistenza al DAO del layer sottostante. I servizi non accedono mai direttamente al database, compongono chiamate al DAO, gestiscono l'eventuale perimetro transazionale dell'uso-caso, aggregano i risultati e li mappano in oggetti del *common.model* da restituire al client.



Le eccezioni SQL vengono intercettate a valle e tradotte in errori di dominio coerenti con il protocollo RMI, i servizi curano inoltre aspetti trasversali come

paginazione/ordinamento nelle ricerche, logging degli eventi significativi e notifiche asincrone verso i client tramite *MonitorInterface* (es. *serverWillStop()* verso i listener). In questo modo il Service Layer fornisce un punto unico di ingresso, sicuro e coerente, alla logica di BookRecommender, schermando il client dai dettagli di persistenza e mantenendo pulite le dipendenze tra moduli.

3.4.4 Presentation Layer



Il Presentation Layer è implementato come applicazione JavaFX e rappresenta il punto di interazione diretto tra l'utente e il sistema. Ogni schermata dell'applicazione corrisponde a un file FXML associato al relativo controller (ad esempio Login, Registrazione, CercaLibro, GestioneLibrerie, DettaglioLibro, ecc.), che gestisce la logica della UI.

Questo livello ha il compito di raccogliere input dall'utente (testi, selezioni, click), validarne il formato minimo e tradurli in chiamate verso il Service Layer remoto tramite i contratti RMI definiti nel modulo common. I controller non contengono logica di business o accesso ai dati, ma delegano queste responsabilità ai servizi remoti, mantenendo così separazione dei ruoli e semplificando la manutenzione.

Per la parte di presentazione sono utilizzati componenti grafici standard JavaFX (TextField, TableView, MenuButton, ecc.) insieme a utility condivise (CliUtil, TableViewEngine, TreeTableEngine) che gestiscono elementi comuni come la costruzione dinamica delle tabelle, la visualizzazione delle valutazioni con icone, la gestione di tooltip e alert.

Il Presentation Layer è anche responsabile dell'aggiornamento in tempo reale dell'interfaccia: i dati restituiti dal Service Layer vengono mappati su oggetti del modello (Libro, Valutazione, Consiglio) e presentati graficamente all'utente in modo ordinato e interattivo. Inoltre, attraverso l'uso di listener registrati su MonitorInterface, la GUI può ricevere notifiche asincrone dal server (ad esempio in caso di shutdown), garantendo un'esperienza coerente e reattiva.

3.5 Strutture dati

Nel client JavaFX la collezione centrale è l'*ObservableList<T>*, utilizzata come modello per *TableView* e *TreeTableView* (es. liste di Libro). Rispetto a una List tradizionale consente binding reattivo: inserimenti, rimozioni e ordinamenti aggiornano la UI a fronte di modifiche ai dati, senza dover scrivere codice aggiuntivo di sincronizzazione tra modello e interfaccia. Per l'elaborazione locale dei risultati di ricerca si usa tipicamente *ArrayList<T>*, l'accesso è indicizzato O(1) e i costi di iterazione minimi.

Nel layer dei servizi e nel DAO prevalgono *List<T>* e *ArrayList<T>* come contenitori di righe mappate da ResultSet. La scelta è naturale perché la cardinalità è variabile e l'uso dominante è la scansione sequenziale, l'overhead di *LinkedList* non porterebbe benefici e viene quindi evitato. Dove serve maggior controllo delle query, compaiono *Map<Chiave, Valore>* o *Map<String, Object>* per convogliare filtri opzionali, quando le chiavi sono finite e note (ad es. un record Libro che rappresenta i risultati di ricerca), l'uso di *Map* risulta più compatto e cache-friendly rispetto a *HashMap*.

Per garantire unicità e deduplicazione client-side (ad es. evitando di proporre lo stesso libro più volte tra risultati o negli "aggiungi a libreria") è impiegato *Set<T>*.

Le classi di dominio condivise in common.model (ad es. Libro, Valutazione, Libro_Details, Token) fungono da DTO serializzabili RMI e, grazie a campi tipizzati, riducono mappe "string-keyed" a favore di oggetti type-safe. Dove un risultato può

manca (es. dettaglio opzionale), l'uso di *Optional<T>* chiarisce l'assenza a livello di API senza ricorrere a null.

Infine, nelle interazioni col DB, le strutture sono intenzionalmente semplici: i DAO restituiscono *List<T>* di modelli mappati, lasciando a indici e vincoli del database la gestione di ordine e unicità. Questa linea guida mantiene basso accoppiamento tra persistenza e presentazione, e massimizza la leggibilità delle query e dei controller.

Le classi ENUM *FXMLtype* e *IMGtype* nel modulo client svolgono un ruolo di centralizzazione e standardizzazione delle risorse dell'interfaccia.

FXMLtype raccoglie in un unico punto i riferimenti ai file FXML che definiscono le diverse viste, rappresentandoli come costanti enumerative. Questo approccio rende più sicuro e leggibile il codice, evitando stringhe “hardcoded” nei controller e garantendo che la navigazione tra schermate avvenga in maniera consistente.

IMGtype svolge la stessa funzione ma per le risorse grafiche (icone, immagini decorative o funzionali). Anche in questo caso l'uso di un ENUM o di una classe dedicata riduce la ridondanza, previene errori di percorso e semplifica l'aggiornamento delle risorse grafiche.

Insieme, queste classi favoriscono manutenibilità e chiarezza del presentation layer, migliorando la gestione delle risorse e riducendo la possibilità di incoerenze nell'interfaccia.

3.6 Design Pattern

- **Layered Architecture:** L'intero sistema è strutturato a livelli (Presentation, Service, DAO, DB), con confini netti tra responsabilità, dipendenze unidirezionali e separazione fra logica UI, logica applicativa e persistenza.
- **MVC (JavaFX):** Le viste FXML (View) con i relativi Controller JavaFX (Controller) orchestrano l'interazione utente, i model del modulo common fungono da Model/DTO mostrati nella UI. Il binding con ObservableList supporta aggiornamenti reattivi.
- **Service Layer + Facade:** Le classi “InterfaceImpl” (es. SearchInterfaceImpl, LibInterfaceImpl, LogRegInterfaceImpl) espongono un punto d'accesso applicativo uniforme verso i casi d'uso, schermando client e controller dai dettagli di persistenza e aggregando operazioni multiple in servizi coerenti.
- **Proxy (RMI):** Il client invoca servizi remoti tramite proxy RMI generati a partire dalle interfacce in common.interfaces. Il proxy intercetta le chiamate locali e le inoltra al server, nascondendo trasporto e serializzazione (Remote Proxy).

- **Data Transfer Object (DTO):** Classi come *Libro*, *Libro_Details*, *Valutazione*, *Token* transitano tra client e server come oggetti serializzabili e tipizzati, minimizzando l'uso di mappe generiche e migliorando la chiarezza del contratto.
- **Observer (notifiche server):** *MonitorInterface* e *ClientListener* implementano un meccanismo push: il server notifica eventi ai client registrati senza polling. È un classico Observer applicato in contesto distribuito. In aggiunta si sarebbe potuto implementare un meccanismo di “ping” periodico dal client verso il server, in modo da verificare proattivamente la disponibilità del servizio ed evitare che l'utente provi a inoltrare richieste quando il server non è più attivo. Tale soluzione avrebbe permesso di prendere provvedimenti immediati (es. disabilitare pulsanti, mostrare un messaggio di disconnessione). Tuttavia, questa feature non è stata implementata per la difficoltà di individuare una tecnica che fosse sempre affidabile ed efficace in ogni contesto di rete e con tutti i possibili stati della JVM RMI. È stato altresì implementato un meccanismo di gestione degli errori durante la comunicazione.
- **Template Method (engines UI):** La presenza di una classe astratta per le tabelle (es. *TableViewEngine*) con metodi “hook” per configurare colonne, placeholder, sorting e azioni. L'algoritmo di costruzione è fisso, i passi variabili sono sovrascrivibili nelle sottoclassi.
- **Singleton (ServerUtil / CliUtil):** Le classi *ServerUtil* e *CliUtil* rappresentano due utility centralizzate che raccolgono procedure comuni rispettivamente lato server (setup, diagnostica delle porte, logging, DAO) e lato client (gestione alert, caricamento FXML, supporto alla UI). A differenza di semplici classi statiche, sono state implementate come veri *Singleton* utilizzando l'idioma *Initialization-on-demand holder*, che garantisce un'istanza unica, creata solo al primo accesso e in maniera intrinsecamente thread-safe. Questo design assicura un punto di accesso globale e coerente a funzionalità condivise e riduce il rischio di duplicazione o inconsistenza. In conclusione, l'uso del Singleton in queste classi contribuisce a mantenere la struttura dell'applicazione più ordinata, affidabile e facilmente estendibile, preservando al tempo stesso chiarezza e manutenibilità del codice.
- **Factory (caricamento viste / risorse):** La combinazione *FXMLtype/loader* e *IMGtype* per le icone standardizza la creazione/risoluzione di risorse, funziona come Factory leggera per ottenere viste e asset senza stringhe sparse, aumenta inoltre la facilità di aggiunta/eliminazione di interfacce.

4. Algoritmi

Le funzionalità di ricerca costituiscono il cuore del sistema, poiché consentono all'utente di recuperare in modo rapido e mirato i libri presenti all'interno del catalogo. Il modulo server implementa algoritmi che, a partire da criteri semplici (titolo, autore, anno), costruiscono query SQL dinamiche e ottimizzate, in grado di restituire i risultati più pertinenti entro i limiti fissati dall'applicazione.

La logica prevede una prima fase di conteggio (`COUNT(*)`) per stimare la cardinalità del risultato e decidere se restituire direttamente l'intero insieme oppure applicare una strategia di ordinamento per rilevanza (basata sulla funzione `STRPOS`) e un limite massimo ai record restituiti. Questa scelta permette di garantire al tempo stesso prestazioni accettabili su dataset ampi (oltre 100.000 libri) e pertinenza dei risultati presentati all'utente.

Dal punto di vista computazionale, le ricerche hanno un costo proporzionale al numero di record corrispondenti: senza indici, l'operazione si traduce in uno scan lineare dell'intera tabella, mentre con indici specializzati (ad esempio GIN trigram su titolo o autore) il costo pratico scende sensibilmente, avvicinandosi al numero effettivo di record recuperati e ordinati.

In questa sezione vengono analizzati i principali algoritmi di ricerca, ovvero quelli in cui la complessità riveste un ruolo fondamentale ai fini della programmazione e dell'ottimizzazione, le query più semplici e di carattere banale non sono invece state oggetto di approfondimento.

4.1 Ricerca per titolo o autore

```
String baseWhere = "FROM LIBRI WHERE LOWER(TITOLO) LIKE LOWER(?)";
String countSql = "SELECT COUNT(*) " + baseWhere;
String dataSql = "SELECT * " + baseWhere;
String orderedAndLimitedSql = "SELECT * " + baseWhere + " ORDER BY
STRPOS(LOWER(?), LOWER(TITOLO))" + "LIMIT (?)";
```

Input: autore del libro, massimo numero di risultati

Output: lista di libri trovati

1. Costruisce la condizione `WHERE LOWER(titolo) LIKE LOWER(?)`.
2. Esegue una query di conteggio (`COUNT(*)`) per stimare il numero totale di risultati.
3. Se i risultati sono pochi, li restituisce direttamente, altrimenti applica un ordinamento per rilevanza usando la funzione `STRPOS` e impone un limite massimo passato dall'utente.

Complessità: Senza indici, la ricerca è $O(N)$ per il filtro più $O(k \log k)$ per l'ordinamento. Con indici trigram e `LIMIT L`, il costo pratico scende a $\sim O(k) + O(L \log L)$ (con $k \approx L$).

Nel caso analizzato si evidenziano le query di ricerca tramite il titolo, complessità e passaggi sono identici per l'autore.

4.2 Ricerca per autore e anno

```
String baseWhere = "FROM LIBRI WHERE LOWER(AUTORE) LIKE LOWER(?) " + "AND  
CAST(ANNOPUBBLICAZIONE AS TEXT) LIKE (?)";  
String countSql = "SELECT COUNT(*) " + baseWhere;  
String dataSql = "SELECT * " + baseWhere;  
String orderedLimitedSql = "SELECT * " + baseWhere + " ORDER BY  
STRPOS(LOWER(AUTORE), LOWER(?))" + " LIMIT (?)";
```

Input: autore, anno di pubblicazione, massimo numero di risultati

Output: lista di libri corrispondenti

1. Applica filtro combinato: WHERE LOWER(autore) LIKE LOWER(?) AND annopubblicazione LIKE ?.
2. Esegue un conteggio preliminare.
3. Se i risultati sono pochi li mostra tutti, altrimenti ordina e limita il numero di record.
4. I risultati vengono convertiti in oggetti Libro.

Complessità: $O(k)$ sul numero di record corrispondenti, $O(k \log k)$ quando è necessario ordinare.

4.3 Ricerche con token utente

```
SELECT DISTINCT l.*  
FROM libri AS l  
JOIN libreria_libro AS ll  
ON ll.idlibro = l.id  
JOIN librerie AS lr  
ON lr.id = ll.idlibreria  
WHERE lr.id_utente = ?  
AND LOWER(l.autore) LIKE LOWER(?);
```

Input: token utente valido, criteri di ricerca (titolo, autore, anno)

Output: lista di libri filtrati in base alle librerie personali

1. Valida il token e recupera l'identificativo dell'utente.
2. Applica il criterio di ricerca (titolo/autore/anno) ma restringe i risultati alle sole librerie associate all'utente (JOIN libreria_libro e librerie).
3. Restituisce la lista finale di libri visibili a quell'utente.

Complessità. Analoga ai casi precedenti, con un costo aggiuntivo minimo dovuto alle join su tabelle indicizzate. Nel caso analizzato si evidenziano le query di ricerca tramite

l'autore, complessità e passaggi sono identici per ricerche tramite titolo e analoghe per autore e anno di pubblicazione.

4.4 Recupero dettagli libro

```
SELECT u.username, v.v_stile, v.c_stile, v.v_contenuto, v.c_contenuto,  
       v.v_gradevolezza, v.c_gradevolezza, v.v_originalita, v.c_originalita,  
       v.v_edizione, v.c_edizione, v.v_finale, v.c_finale  
FROM valutazioni v  
JOIN utenti u ON v.id_utente = u.id  
WHERE v.idlibro = ?
```

```
-----  
SELECT u.username, l.id, l.titolo, l.autore, l.descrizione, l.categoria,  
       l.editore, l.prezzo, l.annopubblicazione, l.mesepubblicazione  
FROM consigli c  
JOIN utenti u ON c.id_utente = u.id  
JOIN libri l ON l.id IN (c.lib_1, c.lib_2, c.lib_3)  
WHERE c.idlibro = ?
```

Input: Libro

Output: Libro_Details(consigli, valutazioni)

1. Valutazioni: join valutazioni v × utenti u su v.id_utente = u.id con filtro v.idlibro = ? mappa liste voti/commenti via utility.
2. Consigli: join consigli c × utenti u e libri l con l.id IN (c.lib_1,c.lib_2,c.lib_3) per c.idlibro = ? aggrega in Hashtable<String, List<Libro>> per username.

Complessità: $O(V + C)$ (V =#valutazioni libro, C =#consigli correlati). Il mapping è tipizzato sui DTO, due query indipendenti, nessuna transazione lunga.

5. Gestione della concorrenza

Nell'applicazione non si è reso necessario introdurre meccanismi espliciti di gestione della concorrenza. Grazie all'auto-commit predefinito offerto da JDBC, ogni richiesta dei client verso il database viene trattata come transazione autonoma e affidata al sistema di gestione della concorrenza del DBMS. Inoltre, al momento della stesura del presente manuale, non sono presenti risorse condivise a livello applicativo tra i diversi client che richiedano ulteriori forme di sincronizzazione.

6. Logging

Il sistema utilizza il logging standard Java (API *java.util.logging*), con livelli coerenti (INFO, WARNING, SEVERE) e messaggi sintetici per eventi operativi (lookup RMI, esecuzione query, errori applicativi/di rete).

6.1 Client

- **Tecnologia:** *java.util.logging* con *ConsoleHandler* (output su *stdout/stderr*).
- **Copertura:** eventi UI significativi, invocazioni ai servizi RMI, gestione eccezioni (es. *RemoteException*), esiti operativi mostrati all'utente via *Alert*.
- **Visibilità:** il logging è visibile solo se l'applicazione viene avviata da terminale (console). Se lanciata con doppio click, la console non è presente e i messaggi non sono visibili all'utente.

6.2 Server

- **Tecnologia:** *java.util.logging* con handler console e inoltro verso una console JavaFX integrata (terminal view) che mostra in tempo reale gli eventi di binding RMI, diagnostica porte e query lato DAO.
- **Copertura:** avvio/stop servizi, esiti rebind, verifiche porte (*ServerUtil*), inizializzazione pool e accesso DB (*DBManager*), eccezioni SQL e timeouts, notifiche verso i client (*Observer*).
- **Visibilità:** presente un terminale JavaFX dedicato che rende i log consultabili anche senza shell esterna.

7. Strumenti, librerie e linguaggi utilizzati

Lo sviluppo dell'applicazione *BookRecommender* ha richiesto l'integrazione di diversi strumenti e tecnologie, selezionati in base alla loro affidabilità, diffusione e aderenza ai requisiti progettuali.

- JDK SE 17.
- IDE: IntelliJ IDEA Ultimate.
- Gluon Scene Builder per la realizzazione delle interfacce grafiche.
- Github per il versioning dell'applicazione e la collaborazione.
- PostgreSQL come DBMS.
- PgAdmin 4 per l'interazione con la base di dati da GUI, testing e controllo.
- Mermaid, draw.io e PlantUML per la realizzazione di diagrammi UML e ER.
- Microsoft Word per la scrittura di manuale tecnico e utente.

Linguaggi adottati:

- Java per la realizzazione effettiva dell'applicazione, sia client che server.
- CSS per lo styling dei componenti della GUI.
- SQL per la realizzazione e interrogazione della base di dati.
- FXML per le View (schermate) della GUI.

Librerie utilizzate:

- PostgreSQL JDBC Driver 42.7.5 per l'interazione lato Java (server) con il database.
- JavaFX (controls, fxml, graphics), versione 17.0.15.
- log4j api 2.0.17 per la gestione dei log d'errore.
- HikariCP versione 5.1.0 per la gestione del pool DBMS.

8. Limiti dell'applicazione e conclusioni

L'applicazione BookRecommender nasce come progetto didattico e, in quanto tale, presenta inevitabilmente alcuni limiti strutturali e progettuali che ne riducono l'applicabilità in contesti produttivi reali. Le scelte implementative sono state guidate più dalla chiarezza didattica e dalla semplicità di gestione che dall'ottimizzazione delle prestazioni o dalla massima scalabilità.

L'architettura client-server basata su RMI offre un modello semplice e funzionale per dimostrare la comunicazione distribuita, ma non è adatta a scenari con un numero elevato di utenti concorrenti. L'assenza di un sistema di load balancing o di un middleware intermedio rende il server un punto di congestione e potenziale collo di bottiglia. Inoltre, la gestione del connection pool è centralizzata e legata a una singola istanza, riducendo la capacità di scalare orizzontalmente.

L'interfaccia client JavaFX e i messaggi applicativi non prevedono internazionalizzazione (i18n). Testi e label sono codificati direttamente nei file FXML o nei controller, rendendo l'applicazione vincolata a una sola lingua. L'eventuale estensione multilingua richiederebbe una revisione completa delle risorse UI con l'introduzione di file di properties dedicati.

Alcune decisioni architetturali sono state assunte per semplicità più che per aderenza alle best practice:

- La distinzione tra entità e DTO non è sempre netta, e alcuni oggetti (es. Libro_Details, Valutazione) avrebbero potuto essere modellati come record in Java 17, ma sono stati mantenuti come classi tradizionali per evitare complicazioni di serializzazione RMI e compatibilità con i DAO.
- L'uso di RMI stesso, pur corretto in un contesto didattico, non rappresenta una tecnologia moderna o largamente utilizzata in scenari enterprise, dove si preferirebbero REST API o gRPC.

- Alcune utility (es. CliUtil, ServerUtil) accentrano responsabilità eterogenee, il che semplifica la gestione ma riduce la coesione e la chiarezza dei ruoli.

Le query di ricerca (LIKE con wildcard, STRPOS per ordinamenti) garantiscono correttezza funzionale ma non sono pensate per dataset molto ampi. In presenza di oltre 100.000 libri, senza indici specifici (es. pg_trgm), il costo delle interrogazioni tende a crescere linearmente. Inoltre, il conteggio preliminare (COUNT(*)) per ogni ricerca comporta un doppio passaggio sul database, con impatto sulle prestazioni complessive. Anche la gestione dei log, lato client e lato server, pur utile a scopo diagnostico, non è ottimizzata per volumi elevati o per una persistenza strutturata dei messaggi.

9. Bibliografia

- Oracle. Java Platform, Standard Edition 17 Documentation. Disponibile su: <https://docs.oracle.com/en/java/javase/17>
- Oracle. JavaFX Documentation. Disponibile su: <https://openjfx.io>
- Oracle. Java Remote Method Invocation (RMI) – Guida ufficiale. Disponibile su: <https://docs.oracle.com/javase/8/docs/technotes/guides/rmi/index.html>
- PostgreSQL Global Development Group. PostgreSQL Documentation (versione 14+). Disponibile su: <https://www.postgresql.org/docs/>
- Brett Wooldridge. HikariCP: High-performance JDBC connection pool. Documentazione ufficiale: <https://github.com/brettwooldridge/HikariCP>
- Oracle. Java Logging Overview (java.util.logging). Disponibile su: <https://docs.oracle.com/javase/8/docs/technotes/guides/logging/>
- GitHub, Inc. GitHub Documentation – Best practices for repositories, branching e collaboration. Disponibile su: <https://docs.github.com>
- Fowler, M. Patterns of Enterprise Application Architecture. Addison-Wesley, 2002.
- Gamma, E., Helm, R., Johnson, R., Vlissides, J. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.
- Freeman, E., Freeman, E. Head First Design Patterns. O'Reilly Media, 2004.